

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;

class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {
```

```
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

        tarjetasValidas.add(t1);

        // Modificación posterior a la inserción
        t1.setNivelAcceso(5);

        System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
        System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
    }
}
```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar `tarjetasValidas.add(t1)`.
- Al ejecutar `t1.setNivelAcceso(5)`.
- Al ejecutar `tarjetasValidas.contains(t1)`.

a) Cuando se ejecuta `tarjetasValidas.add(t1)`, se está agregando una nueva tarjeta de acceso con código "ACC-101" y nivel de acceso 2.

b) Cuando se ejecuta `setNivelAcceso(5)`, se está modificando el nivel de acceso de la tarjeta t1 de 2 a 5. Esta modificación afecta al valor del hashCode y es por eso que la tarjeta no se toma como válida en el conjunto HashSet, ya que el hashCode de la tarjeta ha cambiado y no coincide con el hashCode original que se utilizó para agregarla al conjunto.

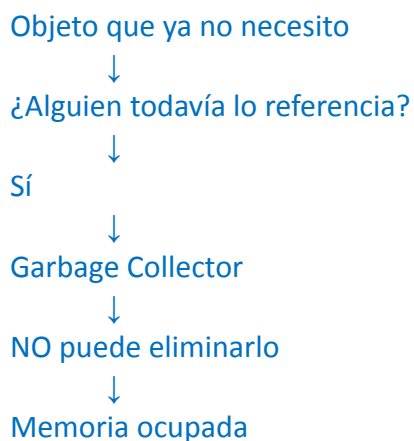
c) Cuando se ejecuta `tarjetasValidas.contains(t1)` se está validando si la tarjeta t1 está presente en el conjunto HashSet. Sin embargo, debido a que el nivel de acceso de la tarjeta ha sido modificado, el hashCode de la tarjeta ya no coincide con el hashCode original que se utilizó para agregarla al conjunto. Por lo tanto, el método `contains()` devuelve false, indicando que la tarjeta no se encuentra en el conjunto, a pesar de que fue agregada previamente.

2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

Queda ubicado valor hashCode de t1: **03** (-1387020803) y Java va a buscarlo al valor hashCode de t1: **00** (-1387020800)

3. **Fuga de Memoria (Memory Leak):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

El Memory Leak ocurre cuando objetos que ya no se necesitan siguen siendo referenciados por estructuras que permanecen activas. Por eso, el Garbage Collector no puede liberarlos, aunque la aplicación ya no los utilice. En un servidor 24/7, estas referencias pueden acumularse progresivamente y consumir cada vez más memoria. El problema puede aparecer por colecciones estáticas, cachés sin límite o referencias que nunca se eliminan.



Se debe a que objetos que ya no deberían necesitar siguen siendo referenciados por alguna estructura que permanece viva.

4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).
- **Inmutabilidad por diseño:** Hacer clases inmutables dejando los campos en `private final` o que la clase no tenga métodos `Set()`.
 - **Claves Basadas en IDs Invariables:** Calculando `equals()` y `hashCode()` sobre un identificador inmutable (ej. `private final Long id`).

Consigna 2: Selección y Diseño de Arquitectura de Colecciones .

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (`java.util`) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones
Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago
Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos
Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz Seleccionada	Implementación concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría	List	LinkedList	O(n)	O(1)	Debido a la

					necesidad de inserción masiva
2. Ofertas	Map	HashMap	O(1)	O(1)	Porque necesitamos almacenar datos en pares de clave - valor, ordenados en este caso por valor (precio)
3. Sesiones	List	ArrayList	O(1)	O(n)	Como la búsqueda será por índice, es conveniente por su complejidad de búsqueda O(1)

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {
    // Algoritmo ineficiente  $O(n^2)$ 
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        boolean duplicado = false;
        for (int j = 0; j < texto.length(); j++) {
            if (i != j && c == texto.charAt(j)) {
                duplicado = true;
                break;
            }
        }
        if (!duplicado) return c;
    }
    return null;
}
```

Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

```
public static Character primerCaracterNoRepetidoRapido(String texto) {
    Map<Character, Integer> repeticiones = new LinkedHashMap<>();
```

```

        // Recorre cada carácter del texto y obtiene la cantidad de repeticiones y las
        guarda en el LinkedHashMap.

        for (int i = 0; i < texto.length(); i++) {

            char caracter = texto.charAt(i);

            repeticiones.put(caracter, repeticiones.getOrDefault(caracter, 0) + 1);

        }

        //Recorre el LinkedHashMap para tomar el primer valor que sea 1, es decir que
        no se repite

        for (Map.Entry<Character, Integer> entrada : repeticiones.entrySet()) {

            if (entrada.getValue() == 1) {

                return entrada.getKey();

            }

        }

        //Total de recorridos: 2.

    }

    return null;

}

```

2. Análisis de Colisiones: Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Ocurre porque la cantidad de objetos posibles en Java es infinita, mientras que los valores de hashCode() están limitados a los números enteros de 32 bits. Por el principio de distribución, inevitablemente dos objetos totalmente distintos terminarán generando el mismo número entero. La estructura lo resuelve dentro del mismo bucket usando equals().

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que s1.hashCode() == s2.hashCode() devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un HashSet<String>, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.

Si `equals()` devuelve `true`: Java determina que `s2` es idéntico en contenido a un objeto que ya está guardado. Por lo tanto, lo desecha. Si `equals()` devuelve `false`: Java comprende que, aunque tengan el mismo código hash, son objetos con datos diferentes. En este caso, lo ingresa en ese mismo cubo.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

En condiciones normales, con una buena distribución de hashes, las operaciones de búsqueda e inserción en un `HashMap` tienen una complejidad promedio de $O(1)$.

Cuando se acumulan muchas colisiones, varios elementos terminan dentro del mismo bucket y Java debe compararlos uno por uno con `equals()` para encontrar el correcto.

- En versiones antiguas de Java (7 o anteriores), cada bucket es una lista enlazada, por lo que en el peor caso (todas las claves colisionando en el mismo bucket) la búsqueda degrada a $*O(n)*$.
- Desde Java 8, si un bucket supera el umbral de 8 elementos (y la tabla tiene capacidad suficiente), esa lista se convierte automáticamente en un árbol rojo-negro, lo que mejora el peor caso a $*O(\log n)*$.